

Ejercicio de práctica – Prueba Kotlin

Sistema de gestión de delivery – FastGo Delivery

Este ejercicio tiene el mismo tipo de requerimientos, complejidad y forma de evaluación que usará la prueba. Resuélvelo en las mismas condiciones (sin apoyo de IA, con los mismos plazos) para que sirva como preparación real.

Condiciones de la solución

- El código debe estar escrito en Kotlin (versión 1.9 o superior).
- Respetar las convenciones de nomenclatura Kotlin: PascalCase para nombres de clase, camelCase para funciones y propiedades.
- El código debe distribuirse en archivos .kt con responsabilidades claramente separadas. Un proyecto de un solo archivo no permite evidenciar modularidad.
- Las secciones con lógica no evidente deben incluir comentarios explicativos. Sin comentarios en secciones clave no es posible evidenciar comprensión.
- El proyecto debe declarar correctamente las dependencias de corrutinas de Kotlin (kotlinx-coroutines-core) en el archivo build.gradle.kts.
- Este es un ejercicio de práctica: resuélvelo sin apoyo de IA ni de terceros para que el resultado refleje tu nivel real antes de la evaluación.

Glosario del sistema

Los siguientes términos se usan de forma consistente en todo el enunciado. Léelos antes de revisar los requerimientos.

- **Pedido:** Unidad que representa una solicitud de entrega. Tiene un código único y un estado.
- **Estado del pedido:** Condición actual de un pedido. Puede estar pendiente, en preparación, en camino o entregado.
- **Repartidor:** Recurso encargado de trasladar los pedidos. Cada repartidor tiene un tipo de vehículo y un estado.
- **Estado del repartidor:** Condición actual de un repartidor. Puede estar disponible, ocupado, en proceso o fuera de servicio.
- **Turno:** Período de operación del sistema desde que se inicializa hasta que se genera el reporte de cierre.
- **Comprobante:** Documento emitido al completarse la entrega. Incluye la distancia recorrida y el monto cobrado.
- **Tarifa base:** Costo por kilómetro antes de aplicar descuentos o recargos, según el tipo de repartidor.
- **Cliente nuevo:** Sin beneficios especiales. Paga la tarifa completa.
- **Cliente frecuente:** Con historial de pedidos en la plataforma. Descuento del 15% sobre la tarifa calculada.
- **Cliente VIP:** Con membresía vigente. Descuento adicional del 40% sobre el monto con IVA.
- **Código de pedido:** Identificador único del pedido. Formato: dos letras, seis dígitos. Ejemplo: DG123456.

Contexto del caso

Sistema de gestión de delivery – FastGo Delivery

FastGo Delivery opera un servicio de reparto a domicilio en zonas urbanas de alta demanda. El área de tecnología ha decidido construir un sistema de consola en Kotlin como núcleo de la lógica de negocio, antes de integrarlo con la aplicación móvil que la empresa tiene en desarrollo.

La empresa necesita un sistema que gestione la asignación de pedidos a repartidores disponibles, aplique tarifas diferenciadas según el tipo de repartidor y el perfil del cliente, y genere reportes al cierre de cada turno. El sistema debe estar preparado para manejar situaciones de error sin interrumpir su funcionamiento.

La asignación de pedidos y la confirmación de entregas implican comunicación con la aplicación móvil de los repartidores. Esa comunicación toma tiempo. Por esta razón, el sistema debe procesar esas operaciones de forma asíncrona: no debe detenerse mientras espera la respuesta de la aplicación.

Importante: el sistema debe permanecer operativo ante cualquier error de datos. Un error no debe detener ni reiniciar el programa.

Requerimientos del sistema

R1 – Tipos de repartidor y reglas de cobro

El sistema debe reconocer tres tipos de repartidor: bicicleta, moto y auto. Cada tipo tiene su propia tarifa base y sus propias reglas de cobro. Los tres tipos comparten un conjunto de datos comunes, pero cada uno determina el monto a cobrar de manera diferente.

Datos comunes a todos los pedidos: *→ clase base*

- Código de pedido en el formato indicado en el glosario. No cambia una vez registrado.
- Distancia a recorrer, en kilómetros. No cambia una vez registrada.
- Fecha y hora exacta de creación del pedido. No cambia una vez registrada.
- Tipo de cliente: (nuevo, frecuente o VIP). No cambia una vez registrado.

Reglas de cobro por tipo de repartidor: *clase base*

- **Bicicleta:** Tarifa base \$300 por km. Si la distancia es inferior a 1 km, se cobra un monto mínimo fijo de \$500, independientemente del tipo de cliente.
- **Moto:** Tarifa base \$500 por km. Si el pedido se marca como **horario peak**, se aplica un recargo del 25% sobre el valor calculado. Este dato se registra al crear el pedido y no cambia.
- **Auto:** Tarifa base \$700 por km. Existe una categoría adicional: **auto refrigerado**, para pedidos que requieren cadena de frío. Si es refrigerado, se aplica un recargo del 20% sobre el valor calculado. Este dato se registra al crear el pedido y no cambia. El detalle que se muestra en pantalla para un auto debe indicar explícitamente si es o no refrigerado.

R2 – Estados de los repartidores

El sistema debe controlar el estado de cada repartidor. Un repartidor solo puede estar en uno de los cuatro estados siguientes:

- **Disponible:** El repartidor está libre. Puede recibir un nuevo pedido. *→ object*
- **Ocupado:** El repartidor tiene un pedido asignado. El sistema debe saber qué pedido es. ●
- **Procesando:** El repartidor está siendo asignado o está confirmando una entrega. El sistema debe indicar el motivo del procesamiento. ●
- **Fuera de servicio:** El repartidor está inhabilitado. El sistema debe registrar el motivo. ●

El sistema debe identificar correctamente el estado actual de un repartidor antes de realizar cualquier operación sobre él. Toda lógica que dependa del estado debe contemplar los cuatro estados posibles. No es correcto asumir que un repartidor siempre estará disponible u ocupado.

R3 – Cálculo de tarifas y validación de datos

El sistema debe calcular el monto total que se cobra por cada pedido entregado. El cálculo siempre sigue este orden:

- Se calcula el costo según la distancia y las reglas del tipo de repartidor (ver R1).
- Se aplica el IVA del 19% sobre el resultado anterior.
- Si el cliente tiene beneficio VIP, se aplica un 40% de descuento sobre el monto con IVA.

Reglas de validación de datos:

- Un código de pedido que no cumpla el formato indicado en el glosario es inválido. El sistema no debe registrar un pedido con código inválido.

Esto se hace con REGEX

- Ningún monto puede ser negativo ni igual a cero. Si el sistema detecta ese resultado, debe reportarlo como un error de datos.
- El tipo de cliente solo puede ser uno de los tres valores definidos en el glosario. Cualquier otro valor es inválido.

Datos del sistema: la empresa se llama "FastGo Delivery" y cuenta con 8 repartidores disponibles por turno. El sistema debe registrar cuánto ha recaudado en total durante el turno y cuánto ha recaudado por cada tipo de repartidor.

R4 – Gestión del catálogo y consultas de negocio

El sistema debe mantener información actualizada sobre los repartidores y los pedidos atendidos durante el turno, y debe ser capaz de responder a las siguientes consultas de negocio:

- ¿Cuántos repartidores están disponibles en este momento?
- ¿Qué pedidos del historial del turno pertenecen a clientes VIP?
- ¿Cuál es el ingreso promedio por pedido entregado en el turno?
- ¿Cuáles son los códigos de todos los pedidos que fueron entregados durante el turno?
- ¿Qué pedido recorrió la mayor distancia durante el turno?

func. de orden superior y métodos no he collections

Reporte de cierre de turno:

Al finalizar el turno, el sistema debe mostrar un resumen por cada pedido entregado: número de comprobante, tipo de repartidor, código de pedido, distancia recorrida y monto pagado. Al final, debe mostrar el total recaudado, la cantidad de pedidos entregados, el ingreso promedio, el tipo de repartidor que más ingresos generó y la cantidad de repartidores disponibles al cierre.

R5 – Registro asíncrono de asignación y entrega

La asignación de pedidos y la confirmación de entregas requieren comunicación con la aplicación móvil de los repartidores. Esta comunicación no es instantánea. El sistema debe modelar ese comportamiento: las operaciones toman tiempo, y el sistema no debe bloquearse mientras las procesa.

Operación de asignación:

- El sistema busca el primer repartidor disponible.
- Mientras espera confirmación de la aplicación, el repartidor queda en estado Procesando con la indicación de que se está asignando el pedido.
- La espera simula la comunicación con la aplicación y dura 2,5 segundos.
- Una vez confirmada la operación, el repartidor cambia a estado Ocupado con el pedido asignado. El sistema informa el resultado en pantalla.

Operación de entrega:

- El sistema localiza al repartidor que tiene asignado el pedido con el código indicado.
- Mientras procesa la entrega, el repartidor queda en estado Procesando con la indicación de que se está confirmando la entrega.
- La espera dura 5 segundos.
- Una vez completada la operación, el sistema emite el comprobante, agrega el pedido al historial, actualiza la recaudación total y libera al repartidor.

En ambas operaciones, el sistema debe actuar según el estado actual del repartidor antes de proceder. No debe, por ejemplo, intentar asignar un pedido a un repartidor que ya está ocupado o fuera de servicio.

R6 – Comportamiento ante errores con try - catch

El sistema debe continuar funcionando cuando ocurra alguna de las siguientes situaciones. En cada caso debe mostrar un mensaje claro y descriptivo en pantalla, sin mostrar trazas técnicas de error.

- **Código de pedido inválido:** Se intenta registrar un pedido con un código que no cumple el formato. El sistema no realiza el registro y notifica el error.

Corru Linz, [Edelay]
Van en el gestor

- **Resultado de tarifa inválido:** El cálculo produce un monto negativo o igual a cero en un caso donde no debería serlo. El sistema notifica el error sin interrumpir la sesión.
- **Pedido no encontrado:** Se intenta confirmar la entrega de un código de pedido que no está asignado a ningún repartidor. El sistema notifica y continúa operando.
- **Sin repartidores disponibles:** Se intenta asignar un pedido, pero no hay repartidores disponibles. El sistema notifica que no hay disponibilidad y no realiza el registro.

El sistema no debe terminar abruptamente por ninguno de estos errores. Después de mostrar el mensaje, debe continuar con las siguientes operaciones normalmente.

R7 – Organización técnica del proyecto

La solución debe distribuirse en archivos Kotlin con responsabilidades claramente separadas. Cada archivo debe contener solo lo que corresponde a su función. No está permitido mezclar responsabilidades entre archivos.

Datos de prueba sugeridos

Los datos siguientes pueden usarse para demostrar el funcionamiento del sistema. No son los únicos datos válidos.

Pedidos a registrar:

- Bicicleta | código: DG100001 | cliente: nuevo | distancia: 0,8 km
- Moto | código: DG100002 | cliente: frecuente | distancia: 5,4 km | horario peak: sí
- Moto | código: DG100003 | cliente: VIP | distancia: 3,0 km | horario peak: no
- Auto | código: DG100004 | cliente: nuevo | distancia: 12,5 km | refrigerado: sí
- Auto | código: DG100005 | cliente: frecuente | distancia: 7,0 km | refrigerado: no
- Prueba de error de código: "12AB3456" — formato inválido.

para probar en el main()